



— WESTERN UNIVERSITY · SDLC

Software Testing

How engineers show that software does what it should, and catch it when it doesn't.

COURSE

SDLC · Chapter 08

INSTRUCTOR

Roeun Mesa

TEAM

OEUN RASMEI
KHOUN KHEYDET

```
~/chapter-08 · test runner

$ run-tests chapter-08
✓ verification matches the spec
✓ validation meets user needs
✗ boundary 99999 + 1 → overflow
🔧 fix and re-run regression suite
✓ 4 passing ready for release
```

— TABLE OF CONTENTS

What this chapter covers

```
describe("Chapter 8: Software Testing")
```

8	Introduction	Verification, validation and what testing can prove	✓
8.1	Development testing	Unit, component and system testing by the developers	✓
8.2	Test-driven development	Write the test first, then the code that passes it	✓
8.3	Release testing	Requirements, scenarios and performance before launch	✓
8.4	User testing	Alpha, beta and acceptance testing with real users	✓
8.5	Conclusion	Why testing earns confidence in software	✓



— CASE STUDIES

Four bugs that made history

1996

FAIL

Ariane 5, Flight 501 \$370M+

Reused Ariane 4 code squeezed a 64-bit velocity value into a 16-bit integer. The faster Ariane 5 overflowed it, and the rocket broke up about 40 seconds after lift-off.

TESTING LESSON

Test boundary values, and re-test reused code in its new setting (8.1)

1999

FAIL

Mars Climate Orbiter \$327.6M

One module reported thruster impulse in pound-force seconds. The module reading it expected newton seconds. The probe drifted off course and burned up at Mars.

TESTING LESSON

Test the interfaces between components (8.1.3)

2012

FAIL

Knight Capital \$440M

New trading code reached 7 of 8 servers. The eighth ran old, retired code that fired millions of unintended orders. The loss took 45 minutes.

TESTING LESSON

Release and regression testing of the whole system (8.2, 8.3)

2024

FAIL

CrowdStrike update 8.5M PCs

A content update gave the sensor 21 input fields when the code expected 20. The out-of-bounds memory read crashed Windows machines at airlines, banks and hospitals.

TESTING LESSON

Test invalid inputs and roll out in stages (8.1.2, 8.4)

— 8 · INTRODUCTION

Testing has two distinct goals

Software testing is an important process in the software development lifecycle.
It checks the software from two directions.

VERIFICATION

**“Are we building
the **product right?**”**

The software should conform to its specification: its functional and non-functional requirements.

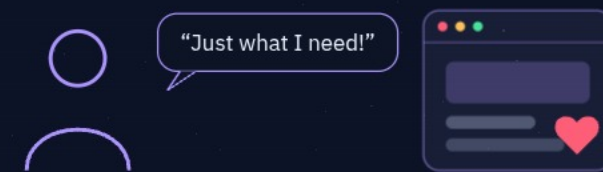


vs

VALIDATION

**“Are we building
the **right product?**”**

The software should do what the customer really requires.



SECTION 8.1

8.1 Development testing

Development testing is the responsibility of the software development team. It happens while the system is being built, and its main job is to find bugs.

Later, a separate team tests the system before it is released to customers. In user testing, customers or system users provide test data and check that tests are successful.

Unit testing

One function, method or object class, tested in isolation.

Component testing

Several units combined, tested through one interface.

System testing

All components integrated and tested as a whole.

STAGE 1 · NOW

Development testing

Development team

STAGE 2

Release testing

Separate testing team

STAGE 3

User testing

Customers and users

— 8.1.1 · DEVELOPMENT TESTING

Unit testing

Unit testing is the process of testing **individual components in isolation**.

A unit can be a single **function** or method, or a whole **object class** with several attributes and methods.

GOAL: COMPLETE TEST COVERAGE OF A CLASS

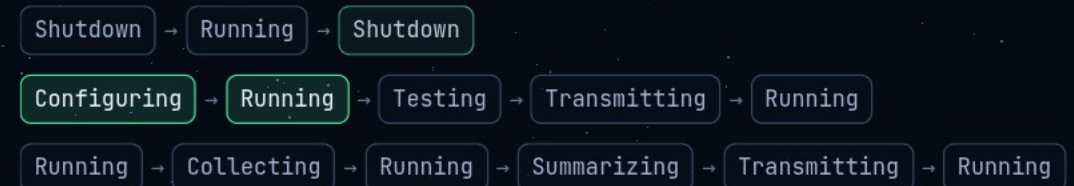
- ✓ Test all operations associated with the object
- ✓ Set and interrogate all object attributes
- ✓ Exercise the object in all possible states

WeatherStation

identifier

- | | |
|--------------------------|---------------------------|
| ✓ reportWeather() | ✓ reportStatus() |
| ✓ powerSave(instruments) | ✓ remoteControl(commands) |
| ✓ reconfigure(commands) | ✓ restart(instruments) |
| ✓ shutdown(instruments) | |

STATE SEQUENCES TO TEST



— 8.1.2 · DEVELOPMENT TESTING

Choosing unit test cases

TWO TYPES OF UNIT TEST CASE

NORMAL Show it works

Test cases that show the component behaves correctly under normal use, for example by demonstrating part of a use case.

```
login("dara", "c0rrect-pw") → "Welcome, Dara"
```

ABNORMAL Expose the bugs

Test cases that expose defects. For example, invalid input should produce an error message and fail gracefully.

```
login("", "") → "Please enter your username"
```

TWO STRATEGIES FOR CHOOSING THEM

Partition testing

Identify groups of inputs that have common characteristics and should be processed in the same way. Choose tests from within each of these groups.

```
ages: [<0 invalid] [0-17 minor] [18-120 adult] [>120 invalid]
```

Guideline-based testing

Use testing guidelines based on the kinds of errors that programmers often make. Choose tests based on these guidelines.

```
empty list · one item · first/middle/last · overflow
```

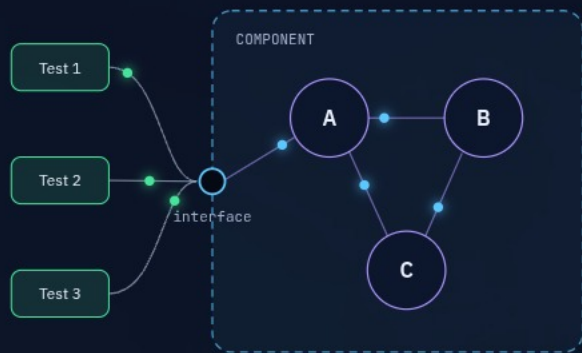
— 8.1.3 - 8.1.4 · DEVELOPMENT TESTING

Component and system testing

8.1.3 · COMPONENT TESTING

Components are made of several **interacting objects**, used through a **defined interface**.

Component testing shows the interface behaves according to its specification, assuming the objects inside have already been unit-tested.



8.1.4 · SYSTEM TESTING

Integrate components to create a version of the system, then **test the integrated system**.

IT CHECKS THAT COMPONENTS

- ✓ are compatible
- ✓ interact correctly
- ✓ transfer the right data at the right time across their interfaces

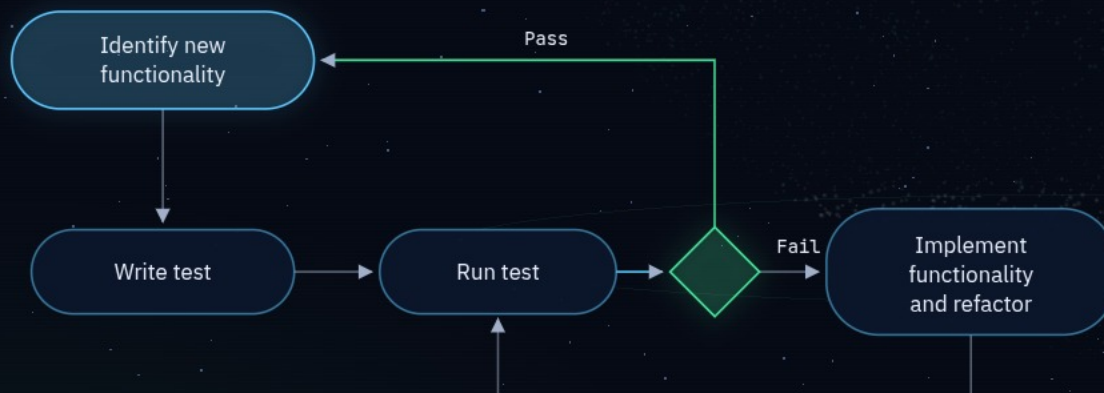
It tests the **interactions between components**, including behaviour that only appears when the parts run together.

Real case: the Mars Climate Orbiter was lost because one module sent pound-force seconds and the other read newton seconds. Each part worked alone; the interface failed.

— 8.2 · TEST-DRIVEN DEVELOPMENT

Write the test first

Test-driven development (TDD) is an approach in which **testing and code development are interleaved** (Beck, 2002). The cycle: red, green, refactor, repeat.



next Move on to the next chunk of functionality and repeat the cycle.

cart.test.js

```
// Feature: orders over $100 get 10% off
test("10% off orders over $100", () => {
  expect(total(120)).toBe(108);
});

function total(amount) {
  return amount > 100 ? amount * 0.9 : amount;
}
```

```
✓ 10% off orders over $100 (2 ms)
Tests: 1 passed
Next: free shipping over $200 →
```

— 8.2 · TEST-DRIVEN DEVELOPMENT

Four benefits of writing tests first



Code coverage

Every piece of code has at least one test, so high coverage means the code is thoroughly tested.



Regression testing

The test suite grows with the code. Re-running it checks that new changes or additions haven't broken anything.

```
let sum = 0;  
for (const i of items)  
  sum += i.price * 0.9;  
return round(sum, 2);
```

Simplified debugging

When a test fails, the problem can usually be pinpointed to the code just written.

- ✓ applies 10% off over \$100
- ✓ rejects negative totals
- ✓ free shipping over \$200
- ✓ rounds to 2 decimals

System documentation

The tests describe what the code is supposed to do, and they never go out of date.

TDD works best for new code. It helps less with large legacy systems and is harder to apply to multithreaded code.

8.3

SECTION 8.3

Release testing



Release testing is the final testing stage before software is released to users.

- Goal:** make sure the software is ready for deployment and meets all user requirements and performance standards.
- Process:** typically a suite of test cases that cover requirements, scenarios and performance.

HOW IT DIFFERS FROM SYSTEM TESTING

	RELEASE TESTING	SYSTEM TESTING
Who	A separate team, not the developers	The development team
Aim	Validation: is it good enough to release?	Defect testing: find bugs
Style	Black-box, from the specification	Can use knowledge of the code

STAGE 1 • DONE

Development testing

Development team

STAGE 2 • NOW

Release testing

Separate testing team

STAGE 3

User testing

Customers and users

Requirements and scenario testing

8.3.1 · REQUIREMENTS-BASED TESTING

Verifies that the software meets each specified requirement. Test cases are designed to cover individual requirements, so each one is checked as correctly implemented.

EXAMPLE · BANKING APP

“Users must be able to transfer money in less than 5 seconds.”

TC-1	Transfer \$50 between own accounts	1.8 s ✓
TC-2	Transfer to another bank	3.1 s ✓
TC-3	Transfer at peak time	4.6 s ✓

8.3.2 · SCENARIO TESTING

Validates complex user workflows end to end, so the software behaves as expected in real-world use. It finds problems that individual functional tests miss.

The tester follows the exact steps in the app, checking each action works smoothly as part of one workflow.

EXAMPLE · SOCIAL MEDIA APP

“Upload a photo, tag friends, add a caption, and share it.”

- ▶ upload a photo
- tag friends
- add a caption
- share it
- scenario passed

— 8.3.3 · RELEASE TESTING

Performance testing

- ◇ **Definition:** evaluates how the software performs under various conditions, such as load, stress and scalability.
- ◇ **Goal:** identify bottlenecks, ensure stability, and guarantee the application performs well under real-world usage.

EXAMPLE · ONLINE GAMING APP

Developers simulate **thousands of players joining a game at the same time** to see if the server can handle the load.
Try it: drag the slider.



— 8.4 · USER TESTING

User testing

Users or customers provide input and advice on system testing, because their real working environment affects a system's **reliability, performance, usability and robustness**.



Alpha testing

Users work with the development team to test the software at the developer's site.

When: developing shrink-wrapped products; users get early news of features.

EARLY BUG DETECTION · LOWER COST



Beta testing

A release of the software is made available to users so they can experiment and raise problems they discover.

When: an early, sometimes unfinished release is ready for evaluation.

REAL-WORLD FEEDBACK · LESS REWORK



Acceptance testing

Customers test a system to decide whether or not it is acceptable. Acceptance implies that payment should be made for the system.

When: the customer formally decides whether to accept the system.

VALIDATES REQUIREMENTS · AGREEMENT TO PAY

— 8.5 • CONCLUSION

Testing turns hope into evidence

Software testing is essential for ensuring a product's **quality, reliability and performance**.

It helps identify and fix issues early, reducing post-release risks and costs.

Through various testing methods, we validate that software **meets requirements and delivers a positive user experience**.

Built into modern practices like **Agile and DevOps**, testing supports faster, higher-quality releases, so organisations can deliver dependable software that meets user needs.

✓ V&V

✓ development

✓ TDD

✓ release

✓ user

5 passing • 0 failing





— CHAPTER 8 • COMPLETE

Thank you!

Questions?

\$ 3 for one question

PRESENTED BY

OEUN RASMEI • KHOUN KHEYDET

INSTRUCTOR

Roeun Mesa